

Week10_예습과제_이재린

≡ 태그

ch5. 토큰화

자연어와 토큰화

1. 자연어

- 자연어란? 사람들이 쓰는 자연히 만들어진 언어(<>인공적으로 만들어진 프로그래밍 언어)
- 자연어 처리(NLP) : 컴퓨터가 인간의 언어를 이해하고 해석 및 생성하기 위한 기술
 - 목표 : 컴퓨터가 인간과 유사한 방식으로 인간의 언어를 이해하고 처리하도록
 - 알고리즘이 해결해야 할 문제 : 모호성(맥락을 이해하고 구분), 가변성(사투리, 강세, 신조어 등), 구조(문장의 구조와 문법적 요소)

then how?

2. 토큰화

- **말뭉치**(자연어 모델을 훈련하고 평가하는데 사용되는 대규모의 자연어)를 일정한 단위인 **토큰**(개별 단어나 문장 부호 등 더 작은 단위)으로 나누어야함.

→ 토큰화

- 입력: '형태소 분석기를 이용해 간단하게 토큰화할 수 있다.'
- 결과: ['형태소', '분석기', '를', '이용', '하', '어', '간단', '하', '게', '토큰', '화', '하', '르', '수', '있', '다', '.']

- 토큰나이저 : 텍스트 문자열을 토큰으로 나누는 알고리즘 또는 SW
 - 공백 분할
 - 정규 표현식 적용
 - 어휘 사전 적용 : 사전에 정의된 단어를 활용 → OOV 존재 가능성 O & 차원의 저주(희소 데이터로 표현될 가능성 O)
 - 머신러닝 활용

단어 및 글자 토큰화

단어 및 글자 토큰화가 비교적 간단하고 명료하지만 꽤 강력함

1. 단어 토큰화

- 단어 토큰화 : 전처리 작업, 텍스트 데이터를 의미 있는 단위인 단어로 분리하는 작업 → 띄어쓰기, 문장 부호, 대소문자 등 **특정 구분자**를 활용

for : 품사 태깅, 개체명 인식, 기계 번역

```
# 5-1. 단어 토큰화
```

```
review="현실과 구분 불가능한 cg, 시각적 즐거움은 최고! 더불어 ost는 더더욱 최고!!"  
tokenized=review.split()  
print(tokenized)
```

```
['현실과', '구분', '불가능한', 'cg,', '시각적', '즐거움은', '최고!', '더불어', 'ost는', '더더욱', '최고!!']
```

→ 공백을 기준으로 나눈다

→ cg와 붙은 마침표처럼 OOV에 취약 & 한국어 접사, 문장 부호, 오타나 띄어쓰기 오류 등에 취약

2. 글자 토큰화

- 글자 토큰화 : 글자 단위로 문장을 나누는 방식, 작은 단어 사전 구축 가능
 - 작은 단어 사전 : 전체 말뭉치 학습시 각 단어를 더 자주 학습 가능
- for : 언어 모델링(다음 문자를 예측)과 같은 시퀀스 예측 작업

[illegible]

→ jamo 라이브러리로 한글 문자열을 자소 단위로 나누어서 반환함.

→ 접사와 문장 부호의 의미 학습 ㄱㄴ

→ but 개별 토큰은 의미 X, 각 토큰의 의미를 조합할 필요가 있음

형태소 토큰화

1. 형태소 토큰화

- 형태소 토큰화 : 텍스트를 형태소 단위로 나누는 토큰화 방법 > 언어의 문법과 구조를 고려해 단어를 분리하고 이를 의미 있는 단위로 분류하는 작업
- 문장 내의 형태소 역할 파악 >> 문장 이해하고 처리

2. 형태소 어휘 사전

- 형태소 어휘 사전 : 각 단어의 형태소 정보를 포함하는 사전 >> 기존의 학습된 어휘 사전에서 새로운 문장이 들어올 때 이를 바탕으로 어휘를 쉽게 학습 ㄱㄴ
 - POS : 품사
 - POS tagging : 각 형태소에 해당하는 품사를 태깅하는 작업

3. KoNLPy

- KoNLPy : 한국어 자연어 처리를 위해 개발된 라이브러리 > 명사 추출, 형태소 분석, 품사 태깅 등의 기능 제공

(1) Okt : SNS 텍스트 데이터 기반으로 개발

```
#5-4. Okt 토큰화
from konlpy.tag import Okt

okt=Okt()
sentence="무엇이든 상상할 수 있는 사람은 무엇이든 만들어 낼 수 있다."

nouns=okt.nouns(sentence)
phrases=okt.phrases(sentence)
morphs=okt.morphs(sentence)
pos=okt.pos(sentence)

print("명사 : ",nouns)
print("구 : ",phrases)
print("형태소 : ",morphs)
print("품사 태깅 : ",pos)

명사 :  ['무엇', '상상', '수', '사람', '무엇', '낼', '수']
구 :  ['무엇', '상상', '상상할 수', '상상할 수 있는 사람', '사람']
형태소 :  ['무엇', '이든', '상상', '할', '수', '있는', '사람', '은', '무엇', '이든', '만들어', '낼', '수', '있다', '.']
품사 태깅 :  [('무엇', 'Noun'), ('이든', 'Josa'), ('상상', 'Noun'), ('할', 'Verb'), ('수', 'Noun'), ('있는', 'Adjective'), ('사람', 'Noun'), ('은', 'Josa'), ('무엇', 'Noun'), ('이든', 'Josa'), ('만들어', 'Verb'), ('낼', 'Noun'), ('수', 'Noun'), ('있다', 'Adjective'), ('.', 'Punctuation')]
```

- 명사 추출, 구문 추출, 형태소 추출, 품사 태깅(튜플 반환)

표 5.1 Okt 품사 태그

태그	품사	태그	품사
Noun	명사	Punctuation	구두점
Verb	동사	Foreign	외국어
Adjective	형용사	Alpha	알파벳
Determiner	관형사	Number	숫자
Adverb	부사	KoreanParticle	자음/모음
Conjunction	접속사	URL	웹 주소
Exclamation	감탄사	Hashtag	해시태그(#)
Josa	조사	Email	이메일
PreEomi	선어말어미	ScreenName	아이디 표기(@)
Eomi	어미	Unknown	미등록
Suffix	접미사		

(2) Kkma : 국립 국어원에서 배포한 세종 말뭉치를 기반으로 학습 → 더 세분화된 품사 태깅 제공

```
# 5-5. 꼬꼬마 토큰화
from konlpy.tag import Kkma

kkma=Kkma()
setence="무엇이든 상상할 수 있는 사람은 무엇이든 만들어 낼 수 있다."

nouns=kkma.nouns(setence)
setences=kkma.sentences(setence)
morphs=kkma.morphs(setence)
pos=kkma.pos(setence)

print("명사 : ",nouns)
print("문장 : ",setences)
print("형태소 : ",morphs)
print("품사 태깅 : ",pos)

명사 :  ['무엇', '상상', '수', '사람', '무엇']
문장 :  ['무엇이든 상상할 수 있는 사람은 무엇이든 만들어 낼 수 있다.']
형태소 :  ['무엇', '이', '든', '상상', '하', 'ㄹ', '수', '있', '는', '사람', '은', '무엇', '이', '든', '만들', '어', '내', 'ㄹ', '수', '있', '다', '.']
품사 태깅 :  [('무엇', 'NNG'), ('이', 'VCP'), ('든', 'ECE'), ('상상', 'NNG'), ('하', 'XSV'), ('ㄹ', 'ETD'), ('수', 'NNB'), ('있', 'VV'), ('는', 'ETD'), ('사람', 'NNG'), ('은', 'JX'), ('무엇', 'NP'), ('이', 'VCP'), ('든', 'ECE'), ('만들', 'VV'), ('어', 'ECD'), ('내', 'VXV'), ('ㄹ', 'ETD'), ('수', 'NNB'), ('있', 'VV'), ('다', 'EFN'), ('.', 'SF')]
```

- 명사, 문장, 형태소, 품사 추출 but 구문 추출 X

표 5.2 꼬꼬마 품사 태그

태그	품사	태그	품사
NNG	보통명사	EPH	존칭 선어말 어미
NNP	고유명사	EPT	시제 선어말 어미
NNB	일반 의존 명사	EPP	공손 선어말 어미
NNM	단위 의존 명사	EFN	평서형 종결 어미
NR	수사	EFQ	의문형 종결 어미
NP	대명사	EFO	명령형 종결 어미
VV	동사	EFA	청유형 종결 어미
VA	형용사	EFI	감탄형 종결 어미
VXV	보조 동사	EFR	존칭형 종결 어미
VXA	보조 형용사	ECE	대등 연결 어미
VCP	긍정 지정사	ECS	보조적 연결 어미
VCN	부정 지정사	ECD	의존적 연결 어미
MDN	수 관형사	ETN	명사형 전성 어미
MDT	일반 관형사	ETD	관형형 전성 어미

태그	품사	태그	품사
MAG	일반 부사	XPN	체언 접두사
MAC	접속 부사	XPV	용언 접두사
JKS	주격 조사	XSN	명사파생 접미사
JKC	보격 조사	XSV	동사 파생 접미사
JKG	관형격 조사	XSA	형용사 파생 접미사
JKO	목적격 조사	XR	어근
JKM	부사격 조사	SF	마침표, 물음표, 느낌표
JKI	호격 조사	SE	출입표
JKQ	인용격 조사	SS	따옴표, 괄호표, 줄표
JC	접속 조사	SP	쉼표, 가운뎃점, 콜론, 빗금
JX	보조사	SO	붙임표(물결, 숨김, 빠짐)
OH	한자	SW	기타기호(논리수학기호, 화폐기호)
OL	외국어	IC	감탄사
ON	숫자	UN	명사추정범주

4. NLTK

- NLTK : 자연어 처리를 위해 개발된 라이브러리 >> 토큰화, 형태소 분석, 구문 분석, 개체명 인식, 감정 분석 기능을 제공

- punkt 모델과 averaged_perceptron_tagger 모델 활용

(1) 영문 토큰화(punkt 모델)

```
# 5-7. 영문 토큰화
from nltk import tokenize

sentence="Those who can imagine anything, can create the impossible."

word_tokens=tokenize.word_tokenize(sentence)
sent_tokens=tokenize.sent_tokenize(sentence)

print(word_tokens)
print(sent_tokens)

['Those', 'who', 'can', 'imagine', 'anything', ',', 'can', 'create', 'the', 'impossible', '.']
['Those who can imagine anything, can create the impossible.']
```

- word_tokenize : 단어 토큰라이저, 공백 기준으로 단어 분리 & 구두점을 처리 >> 단어를 추출해 리스트로 반환
- sent_tokenize : 문장 토큰라이저, 구두점을 기준으로 문장 분리해 리스트로 반환

(2) 영문 품사 태깅 → averaged_perceptron_tagger 모델

```
# 5-8. 영문 품사 태깅
from nltk import tag
from nltk import tokenize

sentence="Those who can imagine anything, can create the impossible."

word_tokens=tokenize.word_tokenize(sentence)
pos=tag.pos_tag(word_tokens)

print(pos)

[('Those', 'DT'), ('who', 'WP'), ('can', 'MD'), ('imagine', 'VB'), ('anything', 'NN'), (',', ','), ('can', 'MD'), ('create', 'VB'), ('the', 'DT'), ('impossible', 'JJ'), ('.', '.')]


```

- pos_tag : 품사 태깅, 앞서 토큰화한 단어들이 들어가야함

표 5.3 Averaged Perceptron Tagger 품사 태그

태그	품사	태그	품사
CC	접속사	PDT	관사
CD	기수(Cardinal Number)	POS	소유격 조사
DT	관형사	PRP	인칭 대명사
EX	there의 대명사형태	PRP\$	소유격 대명사
FW	외래어	RB	부사

태그	품사	태그	품사
JN	전치사	RBR	비교급 부사
JJ	형용사	RBS	최상급 부사
JJR	비교급 형용사	RP	전치사 또는 조동사
JJS	최상급 형용사	VB	기본형 동사
LS	목록 표시	VBD	과거형 동사
MD	조동사	VBG	현재 부사형 동사
NN	단수형 명사	VBN	과거 분사형 동사
NNS	복수형 명사	VBP	1인칭 단수 현재형 동사
NNP	단수형 고유 명사와 서수(Ordinal Number)	VBZ	3인칭 단수 현재형 동사
NNPS	복수형 고유 명사	WDT	관계사
SYM	기호	WP	주격 관계대명사
TO	To 부정사	WP\$	소유격 관계대명사
UH	감탄사	WRB	관계부사

5. spaCy

- 사이언 기반 오픈 소스 라이브러리, 머신러닝 기반의 자연어 처리 라이브러리로 빠른 속도와 높은 정확도가 추구미

```
# 5-9. spaCy 품사 태깅
import spacy

nlp=spacy.load("en_core_web_sm")
sentence="Those who can imagine anything, can create the impossible."
doc=nlp(sentence)

for token in doc:
    print(f"[{token.pos_:5} - {token.tag_:3} : {token.text}]")

[PRON - DT : Those]
[PRON - WP : who]
[AUX - MD : can]
[VERB - VB : imagine]
[PRON - NN : anything]
[PUNCT - , : ,]
[AUX - MD : can]
[VERB - VB : create]
[DET - DT : the]
[ADJ - JJ : impossible]
[PUNCT - . : .]
```

- load() : 모델 설정
- spaCY :객체 지향적으로 구현 > 처리결과를 doc 객체(token으로 구성)에 저장

표 5.4 spaCy 품사 태그

pos_ 속성	tag_ 속성	품사	pos_ 속성	tag_ 속성	품사
ADJ	JJ	형용사	NUM	CD	기수(Cardinal Number)
ADJ	JJR	비교급 형용사	PROPN	NNP	고유 명사
ADJ	JJS	최상급 형용사	PROPN	NNPS	복수형 고유 명사
ADP	IN	전치사	PUNCT	-LRB-	왼쪽 괄호
ADV	RB	부사	PUNCT	-RRB-	오른쪽 괄호
ADV	RBR	비교급 부사	PUNCT	,	쉼표
ADV	RBS	최상급 부사	PUNCT	:	콜론
ADV	WRB	관계 부사	PUNCT	.	마침표, 물음표, 느낌표
AUX	MD	조동사	PUNCT	--	생략 부호
CCONJ	CC	접속사	PUNCT	*	어는 따옴표
DET	DT	관사, 한정사	PUNCT	"	닫는 따옴표
DET	PDT	전치사 한정사	PUNCT	HYPH	하이픈
DET	PRP\$	소유격 대명사	PUNCT	NFP	불필요한 문장 부호
DET	WDT	관계 대명사	SYM	\$	통화
DET	WP\$	소유격 관계 대명사	VERB	VB	동사 기본형
INTJ	UH	감탄사	VERB	VBD	과거형 동사
NOUN	NN	명사	VERB	VBG	동명사, 현재분사
NOUN	NNS	복수형 명사	VERB	VBN	과거분사
PART	POS	소유격 조사	VERB	VBP	동사 현재형
PART	RP	부사적 불변화사	VERB	VBZ	3인칭 단수 동사 현재형

pos_ 속성	tag_ 속성	품사	pos_ 속성	tag_ 속성	품사
PART	TO	To 부정사	X	ADD	이메일
PRON	EX	존재문(there)	X	FW	외래어
PRON	PRP	인칭 대명사	X	LS	목록
PRON	WP	관계 대명사	X	SYM	기호

하위 단어 토큰화

하위 단어 토큰화란

하위 단어 토큰화 : 하나의 단어가 빈번하게 사용되는 하위 단어의 조합으로 나누어 토큰화하는 방법 > 현대의 신조어, 오타자, 축약어 등을 고려하기 위해

예시) 신조어의 기존 형태소 분석기 적용 예시

- 원문: 진짜 멋있네요! 돈줄날만 하네요.
- 결과: ['진짜', '멋있네요', '!', '돈줄날', '만', '하네요', '.']

바이트 페어 인코딩

- 바이트 페어 인코딩 BPE : 다이어그램 코딩, 연속된 글자 쌍이 더 이상 나타나지 않거나 정해진 어휘 사전 크기에 도달할 때까지 조합 탐지와 부호화를 반복 & 자주 등장하는 단어를 하나의 토큰으로 토큰화 <> 덜 등장하는 단어는 여러 토큰의 조합으로 표현

- 원문: abracadabra
- Step #1: AracadAra
- Step #2: ABcadAB
- Step #3: CcadC

→ 빈도수가 높은 입력 데이터는 없는 새로운 글자로 치환하는 과정을 반복

- 토큰나이저일때
 - 빈도 사전: ('low', 5), ('low', 'e', 'r', 2), ('n', 'e', 'w', 'est', 6), ('w', 'i', 'd', 'est', 3)
 - 어휘 사전: ['d', 'e', 'i', 'l', 'n', 'o', 'r', 's', 't', 'w', 'es', 'est', 'lo', 'low', 'ne', 'new', 'newest', 'wi', 'wid', 'widest']

1. 센텐스피스 및 코포라

- 센텐스피스 : 구글에서 개발한 오픈소스 하위 단어 토큰나이저 라이브러리
 - 바이트 페어 인코딩과 유사한 알고리즘으로 입력 데이터 토큰화하고 단어 사전을 생성

- 코포라 : 국립국어원이나 AI Hub에서 제공하는 말뭉치 데이터를 쉽게 사용할 수 있게 제공하는 오픈소스 라이브러리

2. 토큰나이저 모델 학습

- 코포라 라이브러리로 말뭉치 데이터 다운로드 가능(Korpora.load)
- SentencePieceProcessor : 토큰화 수행
- 토큰나이저 모델 학습 : SentencePieceTrainer 클래스의 train으로 토큰나이저 모델 학습

표 5.5 SentencePieceTrainer 매개변수

매개변수	의미
input	말뭉치 텍스트 파일의 경로
model_prefix	모델 파일 이름
vocab_size	어휘 사전 크기
character_coverage	말뭉치 내에 존재하는 글자 중 토큰나이저가 다룰 수 있는 글자의 비율
model_type	토큰나이저 알고리즘 ■ unigram ■ bpe ■ char ■ word
max_sentence_length	최대 문장 길이
unk_id	어휘 사전에 없는 OOV를 의미하는 unk 토큰의 id (기본값: 0)
bos_id	문장이 시작되는 지점을 의미하는 bos 토큰의 id (기본값: 1)
eos_id	문장이 끝나는 지점을 의미하는 eos 토큰의 id (기본값: 2)

>> 결과 : petition_bpe.model(학습된 토큰나이저가 저장된 파일) & petition_bpe.vocab(어휘 사전이 저장된 파일 → 공백도 _로 표현)



'Hello World' → 'Hello_World' → '_Hello+_Wor+ld'(토큰화)

3. 바이트 페어 인코딩 토큰화

- SentencePieceProcessor 클래스로 학습된 모델을 불러와 토큰화 수행

```
#5-13. 바이트 페어 인코딩 토큰화
from sentencepiece import SentencePieceProcessor #토큰화 수행

tokenizer = SentencePieceProcessor()
tokenizer.load("petition_bpe.model")

sentence = "안녕하세요, 토큰나이저가 잘 학습되었군요!"
sentences = ["이렇게 입력값을 리스트로 받아서", "쉽게 토큰나이저를 사용할 수 있습니다."]

tokenized_sentence = tokenizer.encode_as_pieces(sentence) #문장을 토큰화
tokenized_sentences = tokenizer.encode_as_pieces(sentences) #토큰을 정수로 인코딩 >> 정수는 어휘 사전의 토큰에 매핑된 ID(자연어

print("단일 문장 토큰화 :", tokenized_sentence)
print("여러 문장 토큰화 :", tokenized_sentences)

encoded_sentence = tokenizer.encode_as_ids(sentence)
encoded_sentences = tokenizer.encode_as_ids(sentences)

print("단일 문장 인코딩 :", encoded_sentence)
print("여러 문장 인코딩 :", encoded_sentences)

decode_ids = tokenizer.decode_ids(encoded_sentences) # 문자열 데이터 변환
decode_pieces = tokenizer.decode_pieces(encoded_sentences)
print("인코딩 값을 문장으로 디코딩 :", decode_ids)
print("인코딩 값을 토큰으로 디코딩 :", decode_pieces)

단일 문장 토큰화 : ['_안녕하세요', ',', ' ', '_토', '크', '_나', '_이', '_저', '_가', '_잘', '_학', '_습', '_되었', '_군요', '!']
여러 문장 토큰화 : [['_이렇게', '_입', '_력', '_값을', '_리', '_스트', '_로', '_받아서'], ['_쉽게', '_토', '_크', '_나', '_이', '_저', '_를', '_사용할', '_수', '_있', '_답니다', '.']]
단일 문장 인코딩 : [667, 6553, 994, 6880, 6544, 6513, 6590, 6523, 161, 110, 6554, 872, 787, 6648]
여러 문장 인코딩 : [[372, 182, 6677, 4433, 1772, 1613, 6527, 4162], [1681, 994, 6880, 6544, 6513, 6590, 6536, 5852, 19, 5, 2639, 6515]]
인코딩 값을 문장으로 디코딩 : ['이렇게 입력값을 리스트로 받아서', '쉽게 토큰나이저를 사용할 수 있습니다.']
인코딩 값을 토큰으로 디코딩 : ['이렇게 입력값을 리스트로 받아서', '쉽게 토큰나이저를 사용할 수 있습니다.']
```

- 앞서 저장된 petition_bpe.model에서 학습된 모델을 불러옴
- encode로 문장을 토큰화하고 정수로 토큰을 인코딩
- decode로 자연어 처리 모델에서 나온 정수를 문자열 데이터로 변환

4. 어휘 데이터 사전 불러오기

- 어휘 사전을 불러와 딕셔너리 형태로 매핑

```
#5-14. 어휘 사전 불러오기
from sentencepiece import SentencePieceProcessor

tokenizer = SentencePieceProcessor()
tokenizer.load("petition_bpe.model")

vocab = {idx: tokenizer.id_to_piece(idx) for idx in range(tokenizer.get_piece_size())}
print(list(vocab.items())[:5])
print("vocab size :", len(vocab))

[(0, '<unk>'), (1, '<s>'), (2, '</s>'), (3, '니다'), (4, '_아')]
vocab size : 8000
```

- get_piece_size() : 센텐스피스 모델에서 생성도니 하위 단어의 개수 반환
- id_to_piece() : 정숫값을 하위 단어로 변환

워드피스

1. 워드피스란?

- 워드피스 토크나이저 : 빈도 기반이 아닌 **확률 기반**으로 글자쌍을 병합 <> 바이트 페어 인코딩 tn → 학습 과정에서 확률적인 정보를 사용
- 모델이 새로운 하위 단어를 생성할때 : 이전 하위 단어와 함께 나타날 확률을 계산 & 가장 높은 확률을 가진 하위 단어를 선택

수식 5.1 글자 쌍 병합 점수 수식

$$score = \frac{f(x,y)}{f(x)f(y)}$$

- 이때 f는 빈도를 나타내는 함수, f(x,y)는 x,y가 조합된 글자 쌍의 빈도를 의미
 - 빈도 사전: ('l', 'o', 'w', 5), ('l', 'o', 'w', 'e', 'r', 2), ('n', 'e', 'w', 'e', 's', 't', 6), ('w', 'i', 'd', 'e', 's', 't', 3)
 - 어휘 사전: ['d', 'e', 'i', 'l', 'n', 'o', 'r', 's', 't', 'w']
 - 빈도 사전: ('l', 'o', 'w', 5), ('l', 'o', 'w', 'e', 'r', 2), ('n', 'e', 'w', 'e', 's', 't', 6), ('w', 'i', 'd', 'e', 's', 't', 3)
 - 어휘 사전: ['d', 'e', 'i', 'l', 'n', 'o', 'r', 's', 't', 'w', 'id']
 - 빈도 사전: ('lo', 'w', 5), ('lo', 'w', 'e', 'r', 2), ('n', 'e', 'w', 'e', 's', 't', 6), ('w', 'id', 'e', 's', 't', 3)
 - 어휘 사전: ['d', 'e', 'i', 'l', 'n', 'o', 'r', 's', 't', 'w', 'id', 'lo']

>> 단계별로 score가 높은 글자쌍끼리 병합

>> 언제까지? 연속된 글자 쌍이 나타나지 않거나 정해진 어휘 사전 크기에 도달할 때까지

2. 토크나이저스

- 토크나이저스 라이브러리의 워드피스 API로 토크나이저를 구현하고 학습 ㄱㄴ
- 정규화와 사전 토큰화**를 제공
 - 정규화 : 일관된 형식으로 텍스트 표준화 + 모호한 경우를 방지하기 위해 일부 문자를 대체&제거 ex) 공백 제거, 대소문자 변환, 유니코드 정규화, 구두점 처리, 특수 문자 처리
 - 사전 토큰화: bf tokenize 단어와 같은 작은 단위로 나누는 기능 >> 공백 혹은 구두점으로

표 5.6 정규화 클래스

이름	설명
NFD()	NFD 유니코드 정규화
NFKD()	NFKD 유니코드 정규화
NFC()	NFC 유니코드 정규화
NFKC()	NFKC 유니코드 정규화
Lowercase()	입력 문장의 모든 대문자를 소문자로 변환
Strip()	입력 문장의 양 끝에 있는 공백 제거
StripAccents()	모든 액센트 기호 제거
Replace()	문자 혹은 정규 표현식을 기반으로 입력 문장 변환
BertNormalizer()	BERT 모델에 사용된 정규화 적용
Sequence()	여러 개의 정규화 모듈을 병합해 순서대로 수행

표 5.7 사전 토큰화 클래스

클래스	설명
Sequence()	여러 개의 사전 토큰화 모듈을 병합하여 순서대로 수행
ByteLevel()	OpenAI에서 GPT-2를 학습할 때 사용한 방법으로 문자열을 그래픽 문자열로 매핑하고 공백을 기준으로 나눈다.
CharDelimiterSplit()	문자 기준으로 문자열을 나눈다.
Digits()	모든 형태의 숫자 표현을 기준으로 문자열을 나눈다.
Punctuation()	구두점을 기준으로 입력 문자열을 나눈다.
Metaspace(replacement="_")	Whitespace 기준으로 사전 토큰화를 진행하고, replacement로 Whitespace를 치환한다. ▪ 기본값: _(U+2581)
Whitespace()	단어 경계(공백과 구두점)를 기준으로 사전 토큰화를 진행한다. ▪ <code>\w+!([^\w\s])+</code> 정규 표현식을 적용한다.
WhitespaceSplit()	공백 문자를 기준으로 입력 문자열을 나눈다.
Split(pattern, behavior)	정규표현식(pattern)과 동작(behavior)을 기준으로 문자열을 나눈다. ▪ behavior - removed: 삭제 - isolated: 개별 토큰 처리 - merged_with_previous: 이전 문자열과 결합해 토큰 처리 - merged_with_next: 다음 문자열과 결합해 토큰 처리 - contiguous: 다음 문자열이 토큰화되지 않아도 결합해 토큰 처리

- WordPiece 모델을 불러와 정규화(Sequence, NFD, Lowercase)+사전 토큰화 방식 설정
- 모델을 tokenizer.json 파일에서 모델을 불러와 객체 생성, WordPieceDecoder()로 Tokenizer의 디코더를 워드피스 디코더로 설정

ch6. 임베딩

텍스트 벡터화

- 토큰화만으로 모델 학습 불가능 → 텍스트를 숫자로 변환하는 텍스트 벡터화 과정이 필요
- 원-핫 인코딩
 - 각 단어를 index로 매핑해 해당 단어의 idx를 1로 표시하고 나머지는 0으로
 - 단점 : 희소성이 크다.

표 6.1 원-핫 인코딩 매핑 테이블

색인	0	1	2	3
토큰	I	like	apples	bananas

>> 워드 임베딩 기법 ex) Word2Vec. fastText

- 단어를 고정된 길이의 실수 벡터로 표현하는 방법, 단어의 의미를 벡터 공간에서 다른 단어와의 상대적 위치로 표현해 단어 간의 관계를 추론

언어 모델

- 언어 모델 : 입력도니 문장으로 각 문장을 생성할 수 있는 확률을 계산하는 모델 > 주어진 문장을 바탕으로 문맥 이해 & 문장 구성에 대한 예측 수행

자기회귀 언어 모델

- 자기회귀 언어 모델 : 입력된 문장들의 문장들의 조건부 확률을 이용해 다음에 올 단어를 예측
 - (1) 이전에 등장한 모든 토큰의 정보 고려
 - (2) 문장의 문맥 정보를 파악하여 다음 단어 생성

수식 6.1 언어 모델의 조건부 확률

$$P(w_i | w_1, w_2, \dots, w_{i-1}) = \frac{P(w_1, w_2, \dots, w_i)}{P(w_1, w_2, \dots, w_{i-1})}$$

→ 수식 6.1에 조건부 확률의 연쇄법칙을 적용

→ 문장 전체의 확률은 첫 번째 단어의 확률 $P(w_1)$ 과 각 단어가 이전 단어들의 조건부 확률에 따라 발생하는 확률의 곱으로 나타낸다 >> 문장 전체의 확률 계산 가능

수식 6.2 조건부 확률의 연쇄법칙을 적용한 언어 모델의 조건부 확률

$$P(w_i | w_1, w_2, \dots, w_{i-1}) = P(w_1)P(w_2 | w_1) \dots P(w_i | w_1, w_2, \dots, w_{i-1})$$

통계적 언어 모델

- 통계적 언어 모델 : 언어의 통계적 구조를 이용해 문장이나 단어의 시퀀스를 생성하거나 분석한다.
- 마르코프 체인을 이용해 구현 : 이전 상태와 현재 상태 간의 전이 확률로 다음 상태를 예측

수식 6.3 빈도 기반의 조건부 확률

$$P(\text{만나서} | \text{안녕하세요}) = \frac{P(\text{안녕하세요 만나서})}{P(\text{안녕하세요})} = \frac{700}{1000}$$

$$P(\text{반갑습니다} | \text{안녕하세요}) = \frac{P(\text{안녕하세요 반갑습니다})}{P(\text{안녕하세요})} = \frac{100}{1000}$$

- 단점
 - 단어의 순서나 빈도로만 확률 예측 >> 문맥 제대로 파악 X
 - 새로 등장한 단어나 문장에 대하여 정확한 확률 예측하기 힘들 >> 데이터 희소성 문제
- 장점
 - 기존에 학습한 텍스트 데이터에서 패턴을 찾아 확률 분포 생성 → 새로운 문장 생성 가능
 - 다양한 종류의 텍스트 데이터 학습 가능

GPT

- Raw Sentence: GPT는 OpenAI에서 개발한 인과적 언어 모델입니다.
- Input: GPT는 OpenAI에서 개발한
- Prediction-1: 인과적
- Prediction-2: 언어 모델입니다.

BERT

- Raw Sentence: BERT는 Google에서 개발한 마스킹된 언어 모델입니다.
- Input: Bert는 [MASK]에서 개발한 [MASK] 언어 모델입니다.
- Prediction: Google, 마스킹된

N-gram

N-gram

- N-gram 모델 : 텍스트에서 N개의 연속된 단어 시퀀스를 하나의 단위로 취급하여 특정 단어 시퀀스가 등장할 확률을 추정함 >> 토큰 단위가 아닌 N개의 토큰을 묶어서 분석
- N=1 → Unigram, N=2 → Bigram, N=3 → Trigram, N≥4 → N-gram

수식 6.4 N-gram에서의 조건부 확률

$$P(w_t | w_{t-1}, w_{t-2}, \dots, w_{t-N+1})$$

>> $w_{t-1}, w_{t-2}, \dots$: 예측에 사용되는 이전 단어

>> 이전 단어의 개수를 조절해서 모델의 성능 조절 가능

```
#6-1. N-gram
import nltk

def ngrams(sentence, n):
    words = sentence.split()
    ngrams = zip(*[words[i:] for i in range(n)])
    return list(ngrams)

sentence = "안녕하세요 만나서 진심으로 반가워요"

unigram = ngrams(sentence, 1)
bigram = ngrams(sentence, 2)
trigram = ngrams(sentence, 3)

print(unigram)
print(bigram)
print(trigram)

unigram = nltk.ngrams(sentence.split(), 1)
bigram = nltk.ngrams(sentence.split(), 2)
trigram = nltk.ngrams(sentence.split(), 3)

print(list(unigram))
print(list(bigram))
print(list(trigram))

[('안녕하세요',), ('만나서',), ('진심으로',), ('반가워요',)]
[('안녕하세요', '만나서'), ('만나서', '진심으로'), ('진심으로', '반가워요')]
[('안녕하세요', '만나서', '진심으로'), ('만나서', '진심으로', '반가워요')]
[('안녕하세요',), ('만나서',), ('진심으로',), ('반가워요',)]
[('안녕하세요', '만나서'), ('만나서', '진심으로'), ('진심으로', '반가워요')]
[('안녕하세요', '만나서', '진심으로'), ('만나서', '진심으로', '반가워요')]
```

TF-IDF

TF-IDF란

- TF-IDF : 텍스트 문서에서 특정 단어의 중요도를 계산하는 방법 >> 문서 내에서 단어의 중요도를 평가하는데 사용되는 통계적 가중치
- BoW(문서나 문장을 단어의 집합으로 표현하는 방법, 단어의 중복을 허용해 빈도를 기록 <> 원 핫 인코딩, 등장 빈도 고려)에 가중치 부여

표 6.2 BoW 벡터화

	I	like	this	movie	don't	famous	is
This movie is famous movie	0	0	1	2	0	1	1
I like this movie	1	1	1	1	0	0	0
I don't like this movie	1	1	1	1	1	0	0

>> 모든 단어는 동일한 가중치를 갖는다 → 긍정 부정 분류 모델에서 성능이 별로

단어 빈도

- 단어 빈도 : 문서 내에서 특정 단어의 빈도수를 나타내는 값 → 단어의 상대적 중요도 측정 ㄱㄴ but 전문용어나 관용어일수도

수식 6.5 단어 빈도

$$TF(t, d) = count(t, d)$$

문서 빈도

- 문서 빈도 : 한 단어가 얼마나 많은 문서에 나타나는지를 의미 → 특정 단어가 많은 문서에서 단어가 나타나는 횟수 계산

수식 6.6 문서 빈도

$$DF(t, D) = count(t \in d : d \in D)$$

역문서 빈도

- 역문서 빈도 : 전체 문서 수를 문서 빈도로 나누고 로그를 취한 값 → 문서 내에서 해당 단어가 얼마나 중요한지

수식 6.7 역문서 빈도

$$IDF(t, D) = \log\left(\frac{count(D)}{1 + DF(t, D)}\right)$$

TF-IDF

- TF-IDF : 문서 빈도 * 역문서 빈도

수식 6.8 TF-IDF

$$TF-IDF(t, d, D) = TF(t, d) \times IDF(t, d)$$

[사이킷런이 제공하는 TF-IDF 클래스]

```
tfidf_vectorizer = sklearn.feature_extraction.text.TfidfVectorizer(
    input="content",
    encoding="utf-8",
    lowercase=True,
    stop_words=None,
    ngram_range=(1, 1),
    max_df=1.0,
    min_df=1,
    vocabulary=None,
    smooth_idf=True,
)
```

- input : 입력될 데이터의 형태, content는 문자열 데이터 혹은 바이트 형태의 입력값, file object도 가능
- encoding : input값을 받을 때 사용할 텍스트 인코딩값
- lowercase : 소문자 변환 여부
- stop_words : 불용어, 의미없는 단어로 사전에 추가되지 X
- N-gram 범위 : (최소,최대), 해당 형태는 unigram을 의미
- max_df, min_df : 일정 횟수 이상 미만으로 등장하는 단어는 불용어로 처리함을 의미
- vocabulary : 미리 구축한 사전 사용 가능
- smooth_idf : IDF 분모 처리 : 수식과 같이 계산시 분모에 1을 더함

```
#6-2. TF-IDF 계산
from sklearn.feature_extraction.text import TfidfVectorizer

corpus=[
    "That movie is famous movie",
    "I like that actor",
    "I don't like that actor"
]

tfidf_vectorizer=TfidfVectorizer()
tfidf_vectorizer.fit(corpus)
tfidf_matrix=tfidf_vectorizer.transform(corpus)

print(tfidf_matrix.toarray())
print(tfidf_vectorizer.vocabulary_)

[[0.         0.         0.39687454 0.39687454 0.         0.79374908
  0.2344005 ]
 [0.61980538 0.         0.         0.         0.61980538 0.
  0.48133417]
 [0.4804584  0.63174505 0.         0.         0.4804584  0.
  0.37311881]]
{'that': 6, 'movie': 5, 'is': 3, 'famous': 2, 'like': 4, 'actor': 0, 'don': 1}
```